Intelli-Helmet

ABSTRACT

This project presents a Smart Helmet System designed to enhance the safety of motorcycle riders also for Rapido, Swiggy, Zomato riders. The system ensures that the rider wears the helmet, checks for alcohol consumption, and continuously monitors the rider's eye activity to prevent drowsy driving. The ignition of the bike will only start if the rider is wearing the helmet and is not under the influence of alcohol. Additionally, an eye-blink monitoring system alerts the rider using a buzzer when signs of drowsiness are detected. The project uses two Arduino Uno boards one integrated within the helmet and the other on the bike to control ignition along with IR sensors, an alcohol sensor, and a buzzer. The system provides a reliable and cost-effective solution for reducing accidents caused by drunk driving and drowsiness.

G SAI MANOJ GOUD

CONTENTS

ii

Chapter No.	Title	Page No.
1	**INTRODUCTION**	01
	1.1 About the project	01
	1.2 Objectives	02
2	**PROJECT DESCRIPTION**	03
	2.1 Block diagram of project	03
	2.2 Description of block diagram	03
	2.3 Hardware description	03
	2.3.1 Arduino Uno	03
	2.3.2 Mq3 sensor	05
	2.3.3 Buzzer	07
	2.3.4 IR sensor	08
	2.3.5 RF Module	
	2.3.6 Relay Module	
	2.4 Software description	09
3.	**PROJECT IMPLEMENTATION**	12
	3.1 Working	12
	3.2 Results	12
	3.3 Advantages 13 3.4 Disadvantages	13
4.	**CONCLUSION**	
	4.1 Conclusion	14
	4.2 Future scope	14
	BIBLIOGRAPHY	15

G SAI MANOJ GOUD

CHAPTER 1 INTRODUCTION

1.1 INTRODUCTION

Road safety has become a major concern due to the increasing number of accidents involving two-wheeler riders. Many accidents occur because riders do not wear helmets, drive under the influence of alcohol, or fall asleep while riding. To address these issues, the Smart Helmet System integrates multiple safety features to ensure responsible riding behavior. The system checks helmet usage, detects alcohol levels, and monitors the rider's eyes to avoid drowsiness related accidents. By combining sensors and microcontroller technology, this project aims to reduce the chances of fatal accidents and promote safer riding.

1.2 OBJECTIVES

- **To ensure rider safety by verifying mandatory helmet wearing** using an IR sensor before allowing the bike ignition to start.
- **To prevent drunk driving** by detecting alcohol consumption through an alcohol sensor and blocking the bike ignition if alcohol levels exceed the safe limit.
- **To monitor rider drowsiness** using an eye-blink IR sensor and alert the rider with a buzzer when signs of sleepiness are detected.
- **To reduce road accidents** caused by negligence, alcohol consumption, and fatigue through automated safety checks.
- **To design a reliable and cost-effective smart safety system** using Arduino Uno microcontrollers for both helmet and bike units.
- **To promote responsible riding behaviour** by integrating automatic safety features into the riding process.
- **To create a system that continuously monitors rider condition** throughout the journey, ensuring safe driving at all times

G SAI MANOJ GOUD

CHAPTER 2

PROJECT DESCRIPTION

2.1 BLOCK DIAGRAM OF THE PROJECT

As shown in the above schematic diagram it mainly consists of an Arduino, two sensors and a fan and a buzzer. The two sensors are temperature sensor and a gas sensor that is mq2. These sensors are connected to Arduino. Whenever the temperature sensor detects the rise in temperature then immediately it sends an information to arduino, then arduino switches on the fan. And the other sensor is mq2. If any gas leaks then it immediately sends an information to arduino and then it alerts us. The block diagram of the project is shown in fig. 2.1

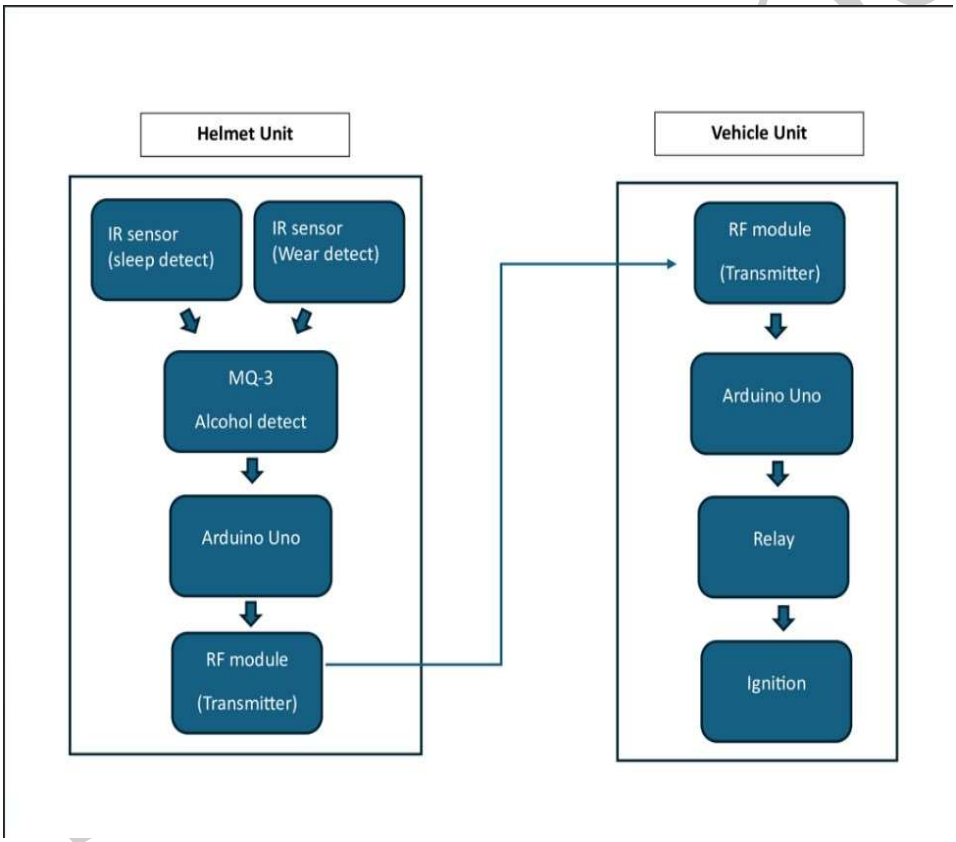


Fig.2.1 Block Diagram

2.3 HARDWARE DESCRIPTION

2.3.1 Arduino UNO

The Arduino Uno is a microcontroller board based on the ATmega328 (datasheet). It has 14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, a 16 MHz ceramic resonator, a USB connection, a power jack, an ICSP header, and a reset button.

It contains everything needed to support the microcontroller; simply connect it to a computer with a USB cable or power it with a AC-to-DC adapter or battery to get started. The Uno differs from all preceding boards in that it does not use the FTDI USB-to-serial driver chip. Instead, it features the Atmega16U2 (Atmega8U2 up to version R2) programmed as a USB-to-serial converter. Revision 2 of the Uno board has a resistor pulling the 8U2 HWB line to ground, making it easier to put into DFU mode. Revision 3 of the board has the

G SAI MANOJ GOUD

following new features: 1.0 pinout: added SDA and SCL pins that are near to the AREF pin and two other new pins placed near to the RESET pin, the IOREF that allow the shields to adapt to the voltage provided from the board. In future, shields will be compatible both with the board that use the AVR, which operate with 5V and with the Arduino Due that operate with 3.3V. The second one is a not connected pin, that is reserved for future purposes. Stronger RESET circuit. Atmega 16U2 replace the 8U2. "Uno" means one in Italian and is named to mark the upcoming release of Arduino 1.0. The Uno and version 1.0 will be the reference versions of Arduino, moving forward. The Uno is the latest in a series of USB Arduino boards, and the reference model for the Arduino platform; for a comparison with previous versions, see the index of Arduino boards.

Microcontroller	ATmega328
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limits)	6-20V
Digital I/O Pins	14 (of which 6 provide PWM output)
Analog Input Pins	6
DC Current per I/O Pin	40 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	32 KB of which 0.5 KB used by bootloader
SRAM	2 KB (ATmega328)
EEPROM	1 KB (ATmega328)

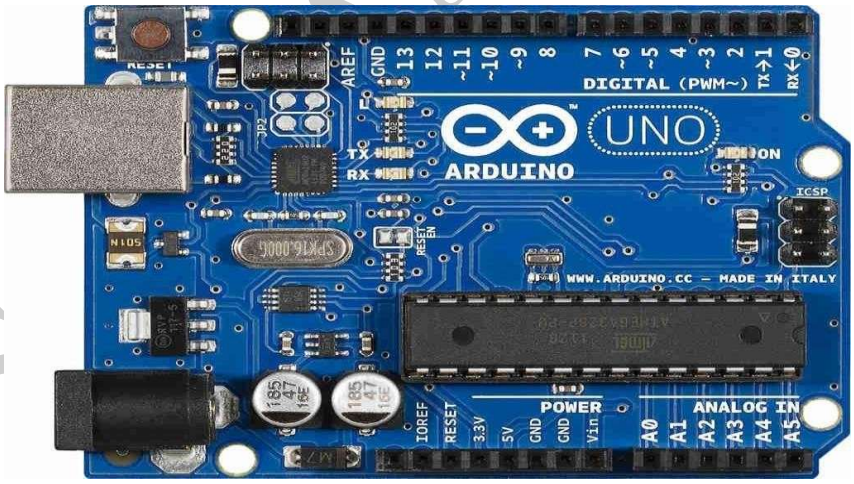


Fig. 2.2 Arduino Uno **Applications:**

- Xoscillo, an open-source oscilloscope
- Arduinome, a MIDI controller device that mimics the Monome
- OBDuino, a trip computer that uses the on-board diagnostics interface found in most modern cars
- Gameduino, an Arduino shield to create retro 2D video games
- ArduinoPhone, a do-it-yourself cellphone

G SRINIVAS GOUD

- Water quality testing platform
- Automatic titration system based on Arduino and stepper motor
- Low cost data glove for virtual reality applications
- Impedance sensor system to detect bovine milk adulteration
- Homemade CNC using Arduino and DC motors with close loop control by Homofaciens
- DC motor control using Arduino and H-Bridge

2.3.2 MQ-3 Sensor

The MQ-3 alcohol sensor is a widely used gas sensor designed specifically for detecting alcohol vapours in the air. The sensitive material of the MQ-3 sensor is SnO_2 (tin dioxide), which exhibits **low conductivity in clean air**. When alcohol vapours are present, the conductivity of the sensor increases proportionally with the concentration of alcohol. Using a simple electronic circuit, this change in conductivity can be converted into an output voltage that corresponds to the alcohol concentration.

The MQ-3 sensor is highly sensitive to **ethanol**, making it suitable for applications such as breath analyzers, ignition interlock systems, and safety helmets. When alcohol molecules reach the sensing layer, they get ionized and interact with the sensor material, causing a measurable change in resistance. Similar to other MQ-series sensors, the MQ-3 requires two voltages: **heater voltage (VH)** to bring the sensor to the correct operating temperature and **test voltage (VC)** to measure the output across the load resistance (RL). The heater ensures stable operation, while VC is used to read the sensor's output (VRL).

The module version of the MQ-3 sensor includes an onboard potentiometer for sensitivity adjustment, allowing calibration based on threshold requirements. It provides both analog and digital outputs, making it compatible with microcontrollers like Arduino. Due to its high sensitivity, fast response time, and low cost, the MQ-3 sensor is ideal for alcohol detection in breath-based systems such as smart helmets, ignition lock mechanisms, and safety monitoring equipment.

Features:

• High sensitivity to alcohol vapors (ethanol)

• Fast response and recovery time

• Stable performance with long life

G SAI MANOJ GOUD

• Operates on 5V DC

• Adjustable sensitivity through onboard potentiometer (module version)

• Suitable for breath analyzers and alcohol detection systems

• Analog output compatible with Arduino ADC

• Operating temperature: –10°C to 50°C

• Heater consumption: < 750 mW

• Interface: Analog / Digital (depending on module)

• Pin Definition: 1–Output, 2–GND, 3–VCC

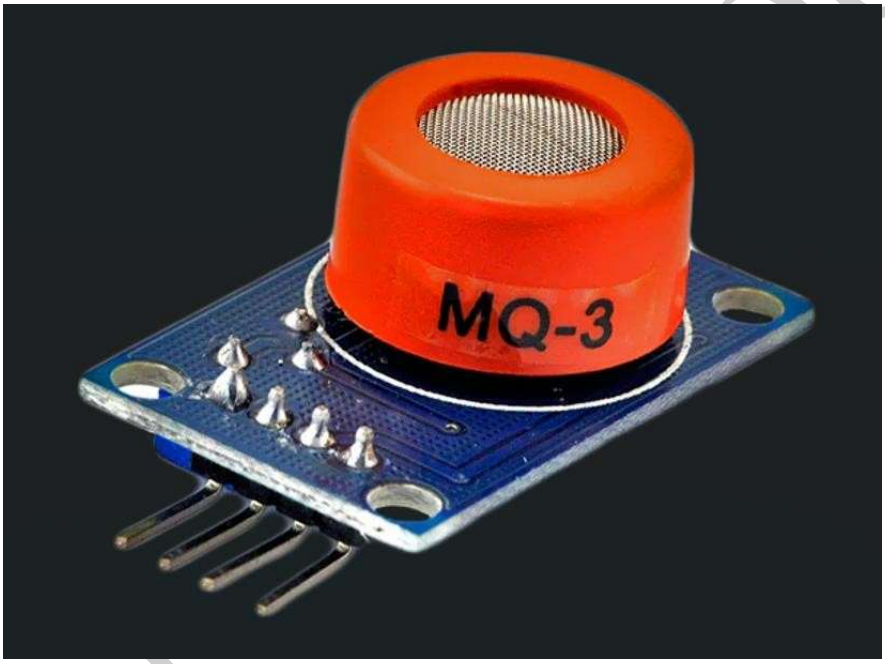


Fig. 2.3 MQ3 sensor

2.3.3 Buzzer

A buzzer or beeper is an audio signaling device, which may be mechanical, electromechanical, or piezoelectric. Typical uses of buzzers and beepers include alarm devices, timers, and confirmation of user input such as a mouse click or keystroke.



Fig.2.4 Buzzer **Types**

of buzzers:

1. Electromechanical

Early devices were based on an electromechanical system identical to an electric bell without the metal gong. Similarly, a relay may be connected to interrupt its own actuating current, causing the contacts to buzz. Often these units were anchored to a wall or ceiling to use it as a sounding board. The word "buzzer" comes from the rasping noise that electromechanical buzzers made. **2. Mechanical**

A joy buzzer is an example of a purely mechanical buzzer. They require drivers.

3. Piezoelectric

A piezoelectric element may be driven by an oscillating electronic circuit or other audio signal source, driven with a piezoelectric audio amplifier. Sounds commonly used to indicate that a button has been pressed are a click, a ring or a beep.

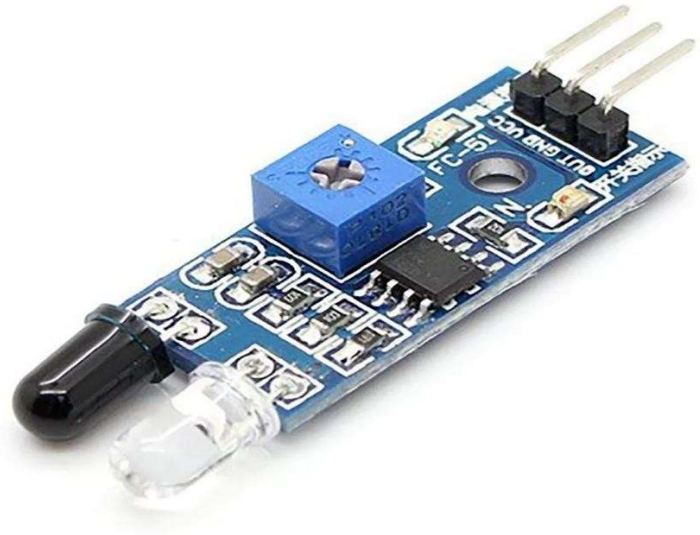
Applications:

While technological advancements have caused buzzers to be impractical and undesirable, there are still instances in which buzzers and similar circuits may be used. Present day applications include:

- Novelty uses
- Judging panels
- Educational purposes
- Annunciator panels
- Electronic metronomes
- Game show lock-out device
- Microwave ovens and other household appliances
- Sporting events such as basketball games

G SAI MANOJ GOUD

2.3.4 IR Sensor



The Infrared (IR) Sensor is an electronic device that is used to detect objects, measure distance, or sense variations in the surrounding environment by using infrared radiation. The sensor operates by emitting IR light from an IR LED and detecting the reflected light using a photodiode or phototransistor. In clean or object-free conditions, the reflected IR light is minimal, resulting in low output. When an object comes within range, the amount of reflected IR light increases, causing a corresponding change in the conductivity of the photodiode. This change is converted into a voltage signal that can be read by a microcontroller such as Arduino.

The IR sensor module is widely used due to its simplicity, low cost, and quick response time. It is commonly employed in obstacle detection, proximity sensing, line-following robots, and safety systems like smart helmets. In the Smart Helmet project, the IR sensor is used for two purposes: detecting whether the rider is wearing the helmet and monitoring the rider's eye-blink activity to identify drowsiness. The sensor requires a small power supply and provides both analog and digital outputs depending on the module used. A potentiometer is included on the module to adjust sensitivity based on application needs.

When the IR beam is interrupted or reflected back to the receiver beyond a set threshold, the output signal changes and is interpreted by the Arduino. This makes the IR sensor a reliable choice for real-time detection tasks in embedded safety systems.

Features:

- Low-cost and easy-to-use detection module
- Fast response and high sensitivity

G SAI MANOJ GOUD

• Adjustable sensitivity using onboard potentiometer
• Suitable for obstacle detection, proximity sensing, and eye-blink detection
• Operates on 3.3V to 5V DC supply
• Digital or Analog output depending on module
• Compact design suitable for wearable systems like smart helmets
• Operating distance: typically 2–30 cm (varies by module)
• Pin Definition: VCC – +5V, GND – Ground, OUT – Output Signal

2.3.5 RF Module

The RF (Radio Frequency) Module is a wireless communication device used to transmit and receive data through radio waves. RF modules typically operate at frequencies such as **315 MHz, 433 MHz, or 2.4 GHz**, depending on the application. An RF system generally consists of two parts: an **RF Transmitter** and an **RF Receiver**. The transmitter takes serial data and sends it wirelessly through electromagnetic waves, while the receiver captures these waves and converts them back into serial data that can be processed by a microcontroller like Arduino.

The RF module works on the principle of amplitude modulation (AM) or frequency modulation (FM), depending on the design. In clean communication conditions, the transmitted signal strength is stable; when obstacles or interference exist, the signal-to-noise ratio may vary, which is handled by the receiver using an internal amplifier and filter circuitry. RF modules are widely used because they are simple, low-cost, and capable of transmitting data over long distances compared to infrared-based systems.

In the Smart Helmet project, the RF module is used to send data wirelessly from the **Helmet Unit** to the **Bike Unit**. The helmet Arduino sends information from sensors—such as helmetwear detection and alcohol detection—via the RF transmitter. The bike Arduino receives this information through the RF receiver and enables or disables the ignition accordingly. RF communication ensures reliable, cable-free data transfer, making the system comfortable and practical for real-world use.

Features:
• Low-cost wireless communication module
• Long-range transmission (up to 100–200 meters depending on environment)
• Operates commonly at 433 MHz

G SAI MANOJ GOUD

• Simple ASK (Amplitude Shift Keying) modulation for easy interface

• Requires only 5V DC power supply

• High receiver sensitivity for stable communication

• Compact size suitable for portable and wearable systems

• Compatible with Arduino through serial communication

• Applications include remote controls, automation systems, wireless alarms, smart helmets, and data-logging devices

Pin Definition (Typical 433 MHz RF Module):

Transmitter:

1 – DATA

2 – VCC (5V)

3 – GND

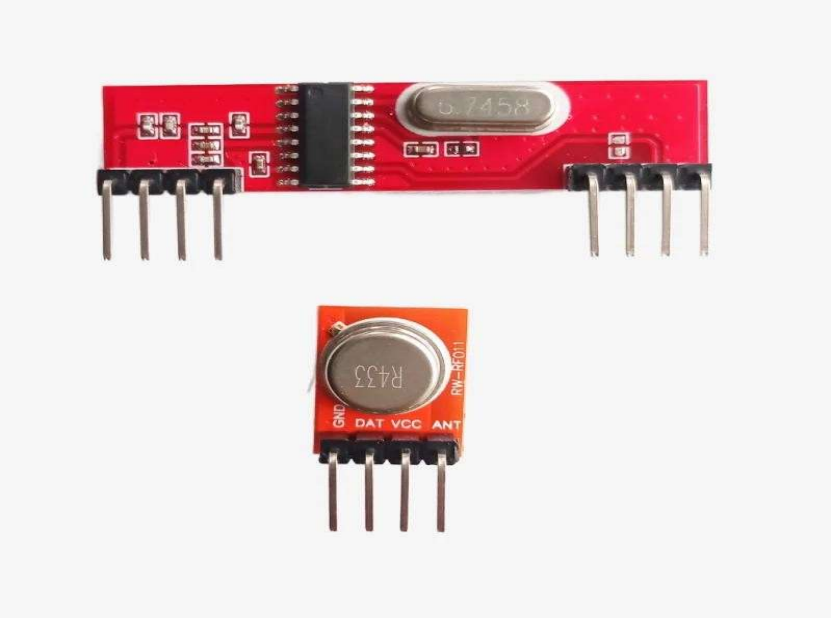
Receiver:

1 – GND

2 – DATA

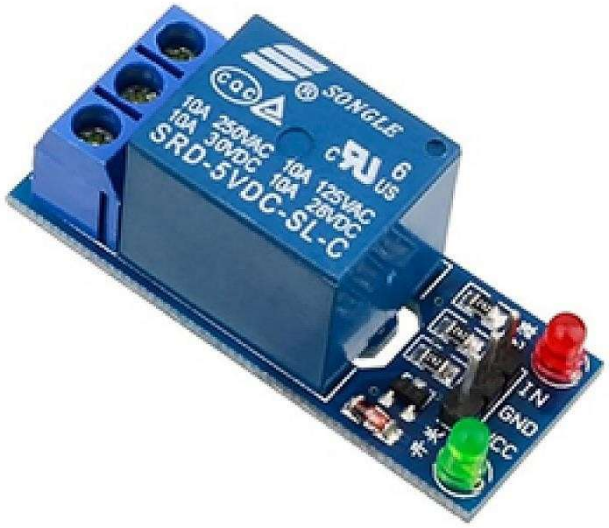
3 – DATA

4 – VCC (5V)



MANOJ GOUD

2.3.5 Relay Module



A relay module is an electrically operated switch used to control high-voltage or high-current loads using a low-power signal from a microcontroller such as Arduino. The relay works on the principle of electromagnetic induction. When a small current flows through the input coil, it generates a magnetic field that pulls a metal armature, thereby opening or closing the connected electrical circuit. In its default state, the relay has two key terminals—Normally Open (NO) and Normally Closed (NC)—which determine whether the external device is activated or deactivated.

Relay modules are widely used in automation, ignition systems, remote switching, and embedded safety applications. In the Smart Helmet project, the relay module plays a crucial role in controlling the bike's ignition system. Based on signals received from the bike-side Arduino, the relay determines whether the ignition should remain off (in case of alcohol detection or helmet not worn) or be allowed to turn on when all safety conditions are satisfied. This makes the relay an essential interface between low-power sensor electronics and high-power bike ignition circuitry.

Most relay modules come with onboard driver transistors, indicator LEDs, isolation components, and protection diodes, making them easy to interface with microcontrollers. They typically operate on 5V DC and provide reliable switching performance suitable for real-time embedded applications.

Features:

- Controls high-voltage devices using low-voltage signals

G SAI MANOJ GOUD

- Electromagnetic switching mechanism
- Supports both Normally Open (NO) and Normally Closed (NC) configurations
- Provides electrical isolation between control circuit and load
- Operates typically at 5V DC input
- Onboard driver circuit for easy microcontroller interfacing
- Indicator LED for switch status
- Long life and reliable operation
- Suitable for ignition control, automation, and safety systems

Pin Definition (Typical 5V Relay Module):

- VCC – +5V supply
- GND – Ground
- IN – Control signal from Arduino
- COM – Common terminal
- NO – Normally Open output
- NC – Normally Closed output

2.4 SOFTWARE DESCRIPTION

The software used here is ARDUINO SOFTWARE:

The Arduino Integrated Development Environment - or Arduino Software (IDE) - contains a text editor for writing code, a message area, a text console, a toolbar with buttons for common functions and a series of menus. It connects to the Arduino and Genuino hardware to upload programs and communicate with them.

Writing Sketches:

Programs written using Arduino Software (IDE) are called sketches. These sketches are written in the text editor and are saved with the file extension in to. The editor has features for cutting/pasting and for searching/replacing text. The message area gives feedback while saving and exporting and also displays errors. The console displays text output by the Arduino Software (IDE), including complete error messages and other information. The bottom righthand corner of the window displays the configured board and serial port. The toolbar buttons allow you to verify and upload programs, create, open, and save sketches, and open the serial monitor.

NB:

G SAI MANOJ GOUD

Versions of the Arduino Software (IDE) prior to 1.0 saved sketches with the extension pde. It is possible to open these files with version 1.0, you will be prompted to save the sketch with the into extension on save.



Verify

Checks your code for errors compiling it.



Upload

Compiles your code and uploads it to the configured board. See uploading below for details.

Note: If you are using an external programmer with your board, you can hold down the "shift" key on your computer when using this icon. The text will change to "Upload using Programmer"



New

Creates a new sketch.



Open

Presents a menu of all the sketches in your sketchbook. Clicking one will open it within the current window overwriting its content.

Note: due to a bug in Java, this menu doesn't scroll; if you need to open a sketch late in the list, use the File | Sketchbook menu instead.



Save

Saves your sketch.



Serial Monitor

Opens the serial monitor.

Additional commands are found within the five menus: File, Edit, Sketch, Tools, and help.

G SAI MANOJ GOUD

Programming on Arduino uno

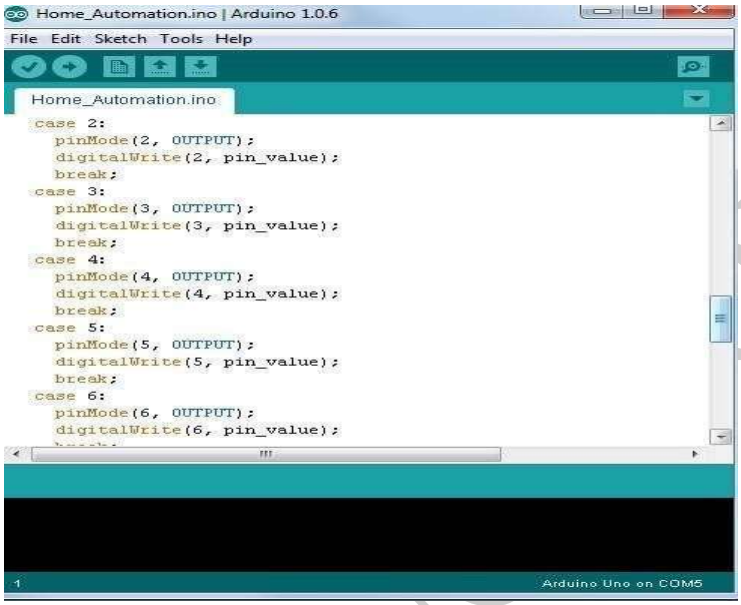


Fig.2.6 Software IDE

In order for the Arduino-Uno board to be able to interact with the application used in this project certain program (code) needs to be uploaded to the Arduino-Uno.

Arduino Company provides user friendly software which allows writing any code for any function wanted to be performed by the Arduino-Uno and upload it to the board.Refer to appendix A for the full source code of the Arduino-Uno board.

G SAI MAHESH

CHAPTER 3 CIRCUIT DIAGRAM AND DESCRIPTION

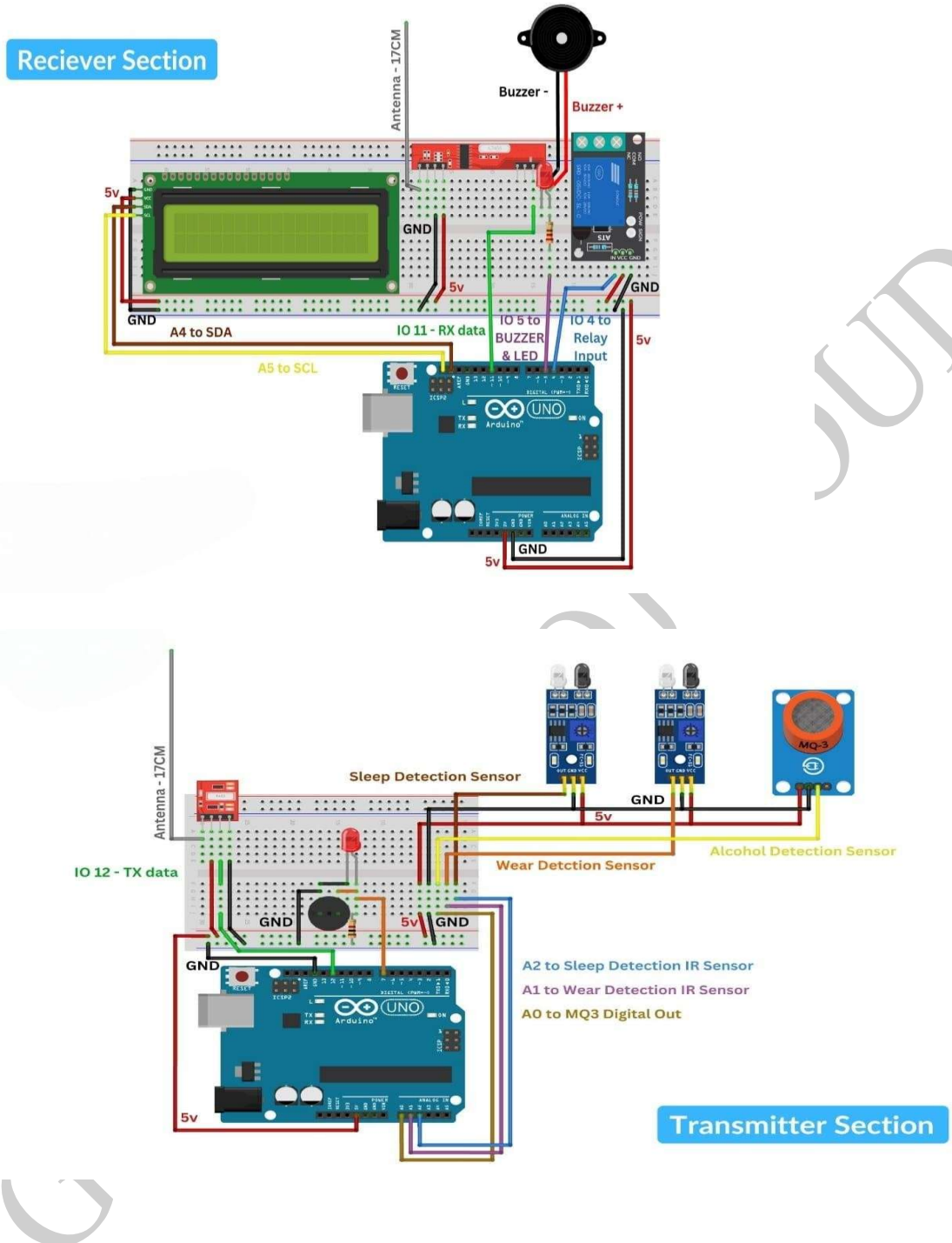
3.1 Working

The Smart Helmet system works by using a combination of sensors placed inside the helmet to determine whether the rider is in a safe condition to operate the motorcycle. When the rider wears the helmet, the IR helmet-detection sensor activates and sends a signal to the helmet Arduino, confirming proper wearing. The alcohol sensor then checks the rider's breath; if alcohol is detected above the threshold level, the bike ignition remains locked, preventing drunk driving. Once both conditions helmet worn and no alcohol detected are satisfied, the helmet Arduino sends a signal to the bike Arduino, allowing the ignition to start.

During the ride, the eye-blink IR sensor continuously monitors the rider's eye activity to detect drowsiness. If the sensor identifies that the rider's eyes are closed for an abnormal duration, the system concludes that the rider might be sleepy. In such cases, the buzzer is activated to alert the rider and prevent potential accidents. Thus, the system ensures continuous safety monitoring by validating helmet usage, blocking ignition when alcohol is detected, and alerting the rider in case of drowsiness. This integrated working mechanism makes the Smart Helmet an effective safety solution for two-wheeler riders.

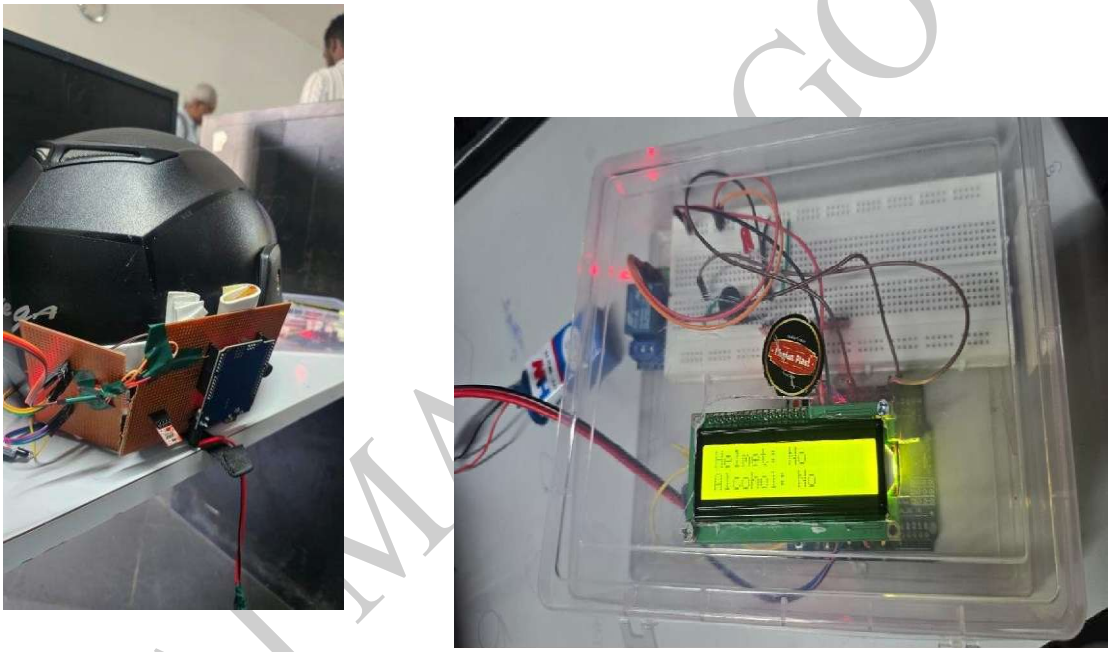
G SAI MANOJ GOUD

Fig.3.1 Schematic diagram



3.2 RESULTS

The experimental result is as shown in below fig. 3.2



3.2 Experimental result

Managed to successfully apply the SMART HELMET USING ARDUINO and it was user friendly and cost effective. User friendly as in anyone can use just a click of a button on an android screen and everything works. And it is cost effective as in it will cost exactly as the project requires (optimum price).

The Smart Helmet system was successfully implemented and tested under various conditions to verify its functionality. The helmet detection IR sensor accurately identified whether the rider

G SAI MA GOUD

was wearing the helmet, and the alcohol sensor reliably detected the presence of alcohol in the rider's breath. The ignition system responded correctly by preventing the bike from starting whenever the helmet was not worn or alcohol was detected, ensuring rider safety.

During continuous riding tests, the eye-blink IR sensor effectively monitored the rider's eye activity and detected prolonged eye closure, indicating drowsiness. In such cases, the buzzer activated immediately, providing a clear alert to the rider. The communication between the helmet unit and the bike unit using Arduino remained stable throughout the tests, and all safety conditions were processed in real time.

Overall, the system performed as expected and successfully demonstrated that integrating alcohol detection, helmet detection, and drowsiness monitoring into a single smart helmet can significantly enhance rider safety and prevent accidents caused by negligence, drunken driving.

3.2 ADVANTAGES

- Prevents drunk driving
- Detects drowsiness
- Ensures helmet wearing
- Reduces accidents
- Simple and cost-effective

3.3 DISADVANTAGES

- Sensor malfunction may cause false detection
- Requires regular maintenance

G SAI MANOJ GOUD

CHAPTER 4 CONCLUSION

4.1 CONCLUSION

The Smart Helmet system successfully demonstrates an effective approach to improving rider safety by integrating multiple safety checks into a single, reliable solution. The project ensures that the bike ignition activates only when the rider is wearing the helmet and is not under the influence of alcohol, thereby preventing two major causes of road accidents. Additionally, the continuous eye-blink monitoring feature helps detect drowsiness during riding and alerts the rider through a buzzer, further enhancing safety.

By combining sensors with Arduino-based control, the system offers a low-cost, practical, and user-friendly method to enforce responsible riding behavior. Overall, the Smart Helmet project proves to be a valuable safety enhancement that can significantly reduce accidents caused by negligence, drunk driving, and fatigue.

4.2 FUTURE SCOPE

The Smart Helmet system can be further enhanced with several advanced features to improve overall rider safety and usability. In the future, the system can be integrated with GPS and accident detection modules to automatically send the rider's location to emergency contacts during accidents. Wireless communication using Bluetooth, Wi-Fi, or GSM can be added to enable seamless interaction between the helmet and bike unit, as well as connectivity with a mobile application for real-time monitoring and alerts. Additional improvements such as AIbased eye-tracking for more accurate drowsiness detection, fall detection sensors, and voice assistance for navigation can greatly enhance both safety and convenience. Solar-powered charging and helmet-fit detection can also be introduced to make the system more efficient, reliable, and user-friendly. Overall, the smart helmet has large potential for future advancements and real-world deployment as an intelligent rider safety solution.

BIBLIOGRAPHY

[11] M. A. Mazharul, S. Ahmed, and H. Rahman, "Design and Implementation of Smart Helmet for Accident Avoidance," *International Journal of Engineering Research & Technology*, vol. 7, no. 3, 2019.

G SAI MANOJ GOUD

[2] N. Shrivastava and R. Kumar, “Alcohol Detection and Vehicle Control System Using Arduino,” *International Journal of Innovative Research in Science, Engineering and Technology*, vol. 6, no. 5, 2018.

[3] T. R. Patil and A. S. Deshpande, “Smart Helmet Using IoT Technology for Accident Prevention,” *International Journal of Advanced Research in Computer and Communication Engineering*, vol. 5, no. 12, 2017.

[4] Arduino.cc, “Arduino Uno Technical Specifications,” Available at: https://www.arduino.cc (Accessed 2024).

[5] MQ-3 Alcohol Sensor Datasheet, Hanwei Electronics Co. Ltd., 2018.

[6] Infrared Sensor Module Datasheet, Vishay Semiconductors, 2020.

[7] RF 433 MHz Transmitter and Receiver Module Datasheet, FS1000A Series, 2019.

[8] S. Gupta and P. Singh, “Drowsiness Detection Using IR Sensor for Vehicle Safety,” *International Journal of Electronics and Communication Technology*, vol. 9, 2020.

APPENDIX

/* **//RECEIVER CODE**

Smart Helmet Reciever code

Features:

- Wireless communication via RF module
(2000 bps, RX pin 11, TX pin 12)

- LCD display (I2C Address: 0x27)

- Ignition control based on helmet status and alcohol
detection

- Buzzer alerts for sleep detection or alcohol presence

- Automatic page switching on LCD every 3 seconds

- Timeout alert when no data is received for 5 seconds

*/

G SAI MANOJ GOUD

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <RH_ASK.h>
#include <SPI.h>

// Initialize LCD display and RF module
LiquidCrystal_I2C lcd(0x27, 16, 2);
RH_ASK rf_driver(2000, 11, 12, 0);

// Define pin connections
const int ignitionRelay =
4; const int buzzerPin = 5;

// Variables to store received data
String str_helmet = "0", str_alcohol = "0",
str_sleep = "0";
int helmetStatus = 0, alcoholLevel = 0,
sleepStatus = 0; bool engineState =
false;

// Timeout and page switching variables
unsigned long lastReceivedTime = 0;
const unsigned long timeout = 5000; int
page = 0, lastPage = -1; unsigned long
```

G SAI MANOJ GOUD

```
lastPageSwitch = 0; const unsigned long
pageInterval = 3000;


void setup() {
  Serial.begin(115200);        //        Initialize        serial
communication
  rf_driver.init(); // Initialize RF module


  pinMode(ignitionRelay,                          OUTPUT);
pinMode(buzzerPin, OUTPUT);
  digitalWrite(ignitionRelay,    LOW);    //    Default:
Vehicle OFF
  digitalWrite(buzzerPin,   LOW);   //   Default:   No
alert


  lcd.init();                  //   Correct   for   this   library
lcd.backlight();


  lcd.setCursor(0,                    0);
lcd.print("Smart               Helmet");
delay(1000);  lcd.clear();


}
```

G SAI MANOJ GOUD

```
void loop() {   uint8_t
buf[20];   uint8_t buflen =
sizeof(buf);

 // Check if RF data is received   if
(rf_driver.recv(buf, &buflen)) {
String str_out = String((char*)buf);
   lastReceivedTime   =   millis();   //   Update   last
received time

   // Extract values from received
string     int comma1 =
str_out.indexOf(',');     int comma2 =
str_out.lastIndexOf(',');

   if   (comma1   >   0   &&   comma2   >   comma1)   {
str_helmet = str_out.substring(0, comma1);
     str_alcohol = str_out.substring(comma1 +
1, comma2);     str_sleep =
str_out.substring(comma2 + 1);
helmetStatus = str_helmet.toInt();
alcoholLevel = str_alcohol.toInt();
sleepStatus = str_sleep.toInt();
```

G SAI MANOJ GOUD

```
    // Control vehicle ignition based on
conditions
    // -------- FIXED ENGINE CONTROL ----
----
      if (helmetStatus == 1 && alcoholLevel
== 0) {           engineState = false;   //
Engine ON
       } else {         engineState = true;
// Engine OFF
     }


     digitalWrite(ignitionRelay,     engineState    ?
HIGH : LOW);


    // Activate buzzer if sleep or alcohol detected
    digitalWrite(buzzerPin,          (sleepStatus          ||
alcoholLevel || page == 99) ? HIGH : LOW);


    Serial.print("Received: ");
    Serial.println(str_out);
  }
 }

 // Handle timeout if no data received for a set
time      if  (millis()  -  lastReceivedTime  >
```

G SAI MANOJ GOUD

```
timeout) {              if (page != 99) {
digitalWrite(buzzerPin, HIGH);
    lcd.clear();        lcd.print("No
Data                    Received");
lcd.setCursor(0,                1);
lcd.print("System        Offline");
page = 99;
  }
return;
 }

 // Handle automatic page switching     if
(millis() - lastPageSwitch > pageInterval) {
lastPageSwitch = millis();      page = (page +
1) % 2;
  delay(200); // Small delay to prevent rapid
switching
 }

 // Clear LCD only when switching pages
if (page != lastPage) {
lcd.clear();   lastPage
= page;
 }
```

G SAI MANOJ GOUD

```
 // Display different pages based on
current page index   switch (page) {
   case 0:  // Page 1: Helmet, Alcohol, and
Sleep  Status             lcd.setCursor(0,  0);
lcd.print("Helmet: ");
     lcd.print(helmetStatus == 1 ? "Yes" : "No
");     lcd.setCursor(0, 1);
lcd.print("Alcohol: ");
     lcd.print(alcoholLevel == 1 ? "Yes" : "No
");
break;


   case 1:  // Page 2: Sleep Status and Engine
State       lcd.setCursor(0, 0);
lcd.print("Sleep: ");       lcd.print(sleepStatus
== 1 ? "Yes" : "No ");       lcd.setCursor(0, 1);
    lcd.print("Ignition: ");
lcd.print(engineState ? "OFF " : "ON");
break;
  }
}
```

//TRNASMITER CODE

```
/*
  Smart Helmet Transmitter Code
```

G SAI MANOJ GOUD

```
Features:
- Wireless transmission of helmet status, alcohol level, and sleep detection
- Drowsiness detection using a sleep sensor
- Buzzer alert for drowsiness detection
- Data sent every 50ms for real-time monitoring
*/

// Include necessary libraries
#include <RH_ASK.h>
#include <SPI.h>

// Debug Mode (Set to true for debugging, false for silent mode)
#define DEBUG_MODE false
// Sensor Pins const int alcoholSensorPin = A0; const int wearSensorPin = A1; const int sleepSensorPin = A2;
const int buzzerPin = 7;  // Buzzer connected to pin 7

// Create RF Object
RH_ASK rf_driver;
```

G SAI MANOJ GOUD

```
// Variables for drowsiness detection
const int drowsinessThreshold = 20;    // Number of sleep signals needed within time window
const unsigned long drowsinessTimeWindow = 10000; // 10 seconds window
const unsigned long resetTime = 5000;  // Reset if no sleep signals within 5 seconds

unsigned long sleepTimestamps[drowsinessThreshold];  // Stores timestamps of detected sleep signals
int sleepIndex = 0; bool isDrowsy = false;

unsigned long lastSleepDetectionTime = 0; unsigned long lastSendTime = 0;
const unsigned long sendInterval = 50; // Send data every 50ms

void setup() {
#if DEBUG_MODE
  Serial.begin(9600);  // Start Serial Monitor
  Serial.println("System Initialized...");
#endif

  rf_driver.init();  // Initialize RF module
pinMode(wearSensorPin, INPUT);
```

G SAI MANOJ GOUD

```
pinMode(alcoholSensorPin, INPUT);
pinMode(sleepSensorPin, INPUT);
pinMode(buzzerPin, OUTPUT);

  digitalWrite(buzzerPin, LOW);  // Ensure buzzer
is OFF initially
}

void  loop()  {        unsigned  long
currentMillis = millis();

  // Read sensor values
  int alcoholLevel = !digitalRead(alcoholSensorPin);
  int sleepSignal = !digitalRead(sleepSensorPin);
  int helmetStatus = !digitalRead(wearSensorPin);

#if DEBUG_MODE
  Serial.print("Helmet: ");
Serial.print(helmetStatus);
  Serial.print(" | Alcohol: "); Serial.print(alcoholLevel);
  Serial.print(" | Sleep Signal: ");
Serial.println(sleepSignal);
#endif
```

G SAI MANOJ GOUD

```
 // **Drowsiness Detection Logic**     if
(sleepSignal) {      lastSleepDetectionTime
= currentMillis;


   // Store the current timestamp in the circular
buffer
   sleepTimestamps[sleepIndex] = currentMillis;
   sleepIndex = (sleepIndex + 1) % drowsinessThreshold;

   // Check if there are 'drowsinessThreshold'
sleep detections within 'drowsinessTimeWindow'
   int count = 0;
   for (int i = 0; i < drowsinessThreshold; i++)
{
     if (currentMillis - sleepTimestamps[i] <= drowsinessTimeWindow)
{
       count++;
     }
   }

   isDrowsy = (count >= drowsinessThreshold);
 }


 // **Reset drowsiness detection if no sleep signals
for resetTime**
```

G SAI MANOJ GOUD

```
 if (currentMillis - lastSleepDetectionTime >
resetTime) {     isDrowsy = false;
 }

#if DEBUG_MODE
 Serial.print("Drowsiness Detected: ");
 Serial.println(isDrowsy ? "YES" : "NO"); #endif

 // **Activate Buzzer on Drowsiness
Detection**   if (isDrowsy) {
   digitalWrite(buzzerPin, HIGH);  // Turn ON
buzzer
 } else {
   digitalWrite(buzzerPin, LOW);   // Turn OFF
buzzer
 }

 // **Send data at regular intervals**
 if (currentMillis - lastSendTime >=
sendInterval) {     lastSendTime =
currentMillis;

   // Convert sensor values to a formatted string
   String data = String(helmetStatus) + "," +
String(alcoholLevel) + "," + String(isDrowsy ?
1 : 0);
```

G SAI MANOJ GOUD

```
   // Convert string to char array for RF
transmission     char msg[data.length() + 1];
data.toCharArray(msg, data.length() + 1);
// Send data via RF
rf_driver.send((uint8_t *)msg, strlen(msg));
rf_driver.waitPacketSent();


#if DEBUG_MODE
    Serial.print("Data Sent: ");
    Serial.println(data);
#endif
  }
  delay(10); // Small delay to avoid overloading
Serial output
}
```

G SAI MANOJ GOUD